

Rapport sur rock, paper, scissors

en JavaScript

Réalisé par :
TAFTAF Wijdane

Encadré par : Pr. Amal Ourdo

Année Universitaire : 2025 / 2026

Table des matières

Introduction	2
Structure du Code	3
Description des Fonctions Principales	4
0.1 Initialisation du Score	4
0.2 Gestion des Événements	4
0.3 Mode Auto Play	5
0.4 Fonction <code>playGame()</code>	5
0.5 Fonction <code>pickComputerMove()</code>	6
Conclusion	8

Introduction

Ce projet consiste à développer un jeu interactif **Pierre–Feuille–Ciseaux** en utilisant le langage **JavaScript**. L'objectif est de permettre à l'utilisateur de jouer contre l'ordinateur, tout en conservant un score persistant grâce à **localStorage**. Le code gère également les interactions clavier, les clics de bouton et un mode **Auto Play**.

Structure du Code

Le code JavaScript est organisé en plusieurs parties principales :

- Initialisation du score et affichage
- Gestion des événements (clics et clavier)
- Fonction `autoPlay()` pour le mode automatique
- Fonction `playGame()` pour la logique du jeu
- Fonction `pickComputerMove()` pour les choix aléatoires

Description des Fonctions Principales

0.1 Initialisation du Score

```
let score = JSON.parse(localStorage.getItem('score')) || {
  wins: 0,
  losses: 0,
  ties: 0
};

updateScoreElement();
```

Cette partie initialise le score à partir du `localStorage` si des données existent déjà, sinon elle crée un objet par défaut.

0.2 Gestion des Événements

```
document.querySelector('.js-rock-button')
  .addEventListener('click', () => {
    playGame('rock');
  });

document.querySelector('.js-paper-button')
  .addEventListener('click', () => {
    playGame('paper');
  });

document.querySelector('.js-scissors-button')
  .addEventListener('click', () => {
    playGame('scissors');
  });
```

Des écouteurs d'événements sont ajoutés pour les boutons du jeu (`rock`, `paper`, `scissors`) ainsi que pour les touches clavier (`r`, `p`, `s`).

```
document.body.addEventListener('keydown', (event) => {
  if (event.key === 'r') {
    playGame('rock');
  } else if (event.key === 'p') {
    playGame('paper');
  } else if (event.key === 's') {
    playGame('scissors');
  }
});
```

0.3 Mode Auto Play

```
let isAutoPlaying = false;
let intervalId; // pour pouvoir l'arrêter

function autoPlay() {
  if (!isAutoPlaying) {
    // démarre le mode auto
    isAutoPlaying = true;
    document.querySelector('.auto-play-button').textContent = 'Stop Auto Play';

    // version avec setTimeout récursif
    function repetition() {
      const playerMove = pickComputerMove(); // joue un coup aléatoire
      playGame(playerMove);
      if (isAutoPlaying) {
        setTimeout(repetition, 1000); // répète chaque seconde
      }
    }

    setTimeout(repetition, 1000);
  } else {
    // arrête le mode auto
    isAutoPlaying = false;
    document.querySelector('.auto-play-button').textContent = 'Auto Play';
  }
}

document.querySelector('.auto-play-button')
  .addEventListener('click', autoPlay);
```

Cette fonction permet de lancer une partie automatique qui simule les choix du joueur toutes les secondes.

0.4 Fonction playGame()

Cette fonction exécute la logique du jeu, met à jour les scores et modifie dynamiquement l'interface graphique.

```
function playGame(playerMove) {  
  const computerMove = pickComputerMove();  
  
  let result = '';  
  
  if (playerMove === computerMove) {  
    result = 'Tie';  
  } else if (  
    (playerMove === 'rock' && computerMove === 'paper') ||  
    (playerMove === 'paper' && computerMove === 'scissors') ||  
    (playerMove === 'scissors' && computerMove === 'rock')  
  ) {  
    result = 'You lose';  
  } else {  
    result = 'You win';  
  }  
  
  if (result === 'You win') {  
    score.wins += 1;  
  } else if (result === 'You lose') {  
    score.losses += 1;  
  } else {  
    score.ties += 1;  
  }  
  
  localStorage.setItem('score', JSON.stringify(score));  
  updateScoreElement();  
  
  document.querySelector('.js-result').innerHTML = result;
```

```
document.querySelector('.js-result').innerHTML = result;  
  
document.querySelector('.js-moves').innerHTML = `  
  You  
    
    
  Computer  
`;  
;  
}  
  
function updateScoreElement() {  
  document.querySelector('.js-score')  
    .innerHTML = `Wins: ${score.wins}, Losses: ${score.losses}, Ties: ${score.ties}`;  
}
```

0.5 Fonction pickComputerMove()

Cette fonction détermine aléatoirement le choix de l'ordinateur à chaque partie. Le score du joueur est sauvegardé grâce à l'API `localStorage`, permettant de conserver les résultats même après rechargement de la page. L'interface affiche les résultats, les icônes des coups joués et les scores cumulés.

```
function pickComputerMove() {  
  const randomNumber = Math.random();  
  
  let computerMove = '';  
  
  if (randomNumber >= 0 && randomNumber < 1 / 3) {  
    computerMove = 'rock';  
  } else if (randomNumber >= 1 / 3 && randomNumber < 2 / 3) {  
    computerMove = 'paper';  
  } else if (randomNumber >= 2 / 3 && randomNumber < 1) {  
    computerMove = 'scissors';  
  }  
  
  return computerMove;  
}
```


Conclusion

Ce tp démontre l'utilisation pratique du JavaScript pour créer un mini-jeu interactif avec gestion d'événements, DOM dynamique et stockage local. Il permet de se familiariser avec la manipulation du navigateur et l'organisation d'un code modulaire.